ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

We introduce a lifecycle audit for determining whether score gains from self-evolving agent skills represent retained skill improvement. A post-revision score alone is insufficient because optimizer variation, validation selection, execution randomness, and verifier choice can improve the endpoint without creating a durable artifact effect. We combine repeated evaluations, controlled task variations, and lifecycle interventions to examine whether improvements from skill evolution persist and transfer, and to trace their origins to feedback, revision, selection, skill content, and evaluation. The audit separates five evidential claims: selected gain, retained artifact effect, bounded transfer, evolution-pathway dependence, and measurement dependence. This claim-grounded framework lets self-evolving systems report what their experiments establish without equating a better benchmark score with reusable skill learning.

***Index Terms*—** Agent skills, self-evolution, lifecycle audit, skill evaluation, generalization

1. INTRODUCTION

Persistent agent skills offer an appealing route to continual improvement. Instead of updating model parameters, an agent can revise an external artifact that stores reusable instructions, scripts, references, templates, or other supporting resources. Recent systems generate, repair, select, and retain such artifacts from execution traces and evaluator feedback, often reporting higher post-evolution benchmark scores as evidence that the agent has learned a better skill [1–4].

A higher score is not, by itself, an identified skill effect. The selected run may be unusually favorable; the optimizer may produce variable candidates; a validation gate may retain a narrow patch; repeated execution may eventually succeed without a durable behavioral change; and a verifier may reward only part of the intended behavior. These possibilities do not imply that self-evolution is ineffective. They imply that an endpoint score alone cannot locate where the improvement resides.

We therefore ask two connected questions. When self-evolution raises the benchmark score, does the revised skill itself retain a behavioral improvement under fresh deployment and controlled task variation? Regardless of whether it does, which parts of the evolution-and-evaluation lifecycle produced the observed gain?

We organize the evidence as a diagnosis of utility, behavioral scope, and gain attribution. Standard-task evaluations of SkillEvolver, SPARK, and CoEvoSkills first establish the observed utility of completed artifacts across two runs. Controlled task changes then reveal different scopes of usefulness, and artifact and trace inspection examines the decisions preserved during reuse. This motivates the next diagnostic step: locating an improvement in the artifact's evolution history. Complementary studies of SkillRevise, SkillHone, and SkillOpt use retention tests and lifecycle interventions to examine trace-conditioned revision, history-based iterative honing, and validation-gated editing. Each study addresses a specific part of the identification question.

Our audit follows the lifecycle from feedback to revision, selection or adoption, frozen artifact deployment, and measurement. We separate historical success from fresh retention, final-skill utility from actual parent-to-final revision effects, and same-task retention from controlled transfer. We then intervene on feedback, optimizer sampling, gate provenance, artifact components, and verifier choice. A short information-integrity check bounds direct answer or oracle exposure as an alternative explanation; it is not the paper's organizing hypothesis.

The resulting picture is deliberately more qualified than either *self-evolution works* or *self-evolution only exploits the evaluator*. Revised artifacts can cause real behavioral improvements, but the demonstrated effects are often narrow. Single-run or selected gains need not survive fresh deployment. Useful traces and feedback must be converted into an effective edit, adopted into the persistent artifact, and retained behaviorally. Finally, a better score can reflect compatibility with a measurement surface rather than broader task behavior.

Our central contribution is a claim-grounded lifecycle audit: each reported gain is matched to the intervention required to support it and to the strongest conclusion that intervention permits. We make three contributions:

- **Identification framework.** We separate a reported gain from retention, behavioral scope, evolution-and-selection pathway, and measurement dependence.
- **Lifecycle audit.** We apply matched interventions across three representative skill-evolution mechanisms without turning heterogeneous studies into a method leaderboard.
- **Reporting protocol.** We specify the lineage, contrasts, and artifacts required to support each claim level, from selected gain to bounded transfer.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback that may include a verifier result, a gold answer, a diagnosis, or evaluator feedback. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

自进化代理技能是否在学习技能?

匿名提交

摘要

我们介绍了一种生命周期审计方法，用于判断自进化代理技能带来的分数提升是否代表技能的持续改进。仅凭修订后的分数是不够的，因为优化器变异、验证选择、执行随机性和验证器选择都可能在不创建持久性工件效应的情况下提高最终结果。我们结合重复评估、受控任务变异和生命周期干预，来检验技能进化带来的改进是否持久且可迁移，并追踪其来源到反馈、修订、选择、技能内容和评估。该审计将证据主张分为五类：选择增益、持久工件效应、有界迁移、进化路径依赖和测量依赖。这一基于主张的框架允许自进化系统报告其实验结果，而不将更好的基准分数等同于可重用的技能学习。

索引词— 代理技能、自进化、生命周期审计、技能评估、泛化

1. 引言

持久的代理技能为持续改进提供了一条有吸引力的途径。代理无需更新模型参数，而是可以修订存储可重用指令、脚本、参考、模板或其他支持资源的的外部工件。最近的系统从执行轨迹和验证器反馈中生成、修复、选择和保留此类工件，通常报告更高的进化后基准分数，以此证明代理学会了一种更好的技能 [1–4]。

更高的分数本身并不是一种已识别的技能效果。选定的运行可能特别有利；优化器可能产生可变候选；验证门可能保留一个狭窄的补丁；重复执行最终可能在没有持久行为变化的情况下成功；验证器可能只奖励预期行为的一部分。这些可能性并不意味着自我进化是无效的。它们意味着仅凭终点分数无法定位改进所在。

因此，我们提出两个相关的问题。当自我进化提高基准分数时，修订后的技能本身在新的部署和控制任务变化下是否保留行为改进？无论它是否这样做，哪些进化与评估生命周期中的部分产生了观察到的收益？

我们将证据组织为效用、行为范围和收益归因的诊断。SkillE-volver、SPARK和CoEvoSkills的标准任务评估首先建立完成工件在两次运行中的观察效用。受控任务变化随后揭示不同用途范围，而工件和轨迹检查则检验在重用过程中保留的决策。这促使我们进行下一步诊断：定位工件进化历史中的改进。SkillRevise、SkillHone和SkillOpt的补充研究使用保留测试和生命周期干预来检验轨迹条件下的修订、基于历史的迭代精炼和验证门控编辑。每项研究都针对识别问题的特定部分。

我们的审计遵循从反馈到修订、选择或采纳、冻结工件部署和测量的生命周期。我们将历史成功与新鲜保留、最终技能效用与实际父到最终修订效应、相同任务保留与受控迁移区分开来。然后我们对反馈、优化器采样、门控溯源、工件组件和验证器选择进行干预。简短的信息完整性检查将直接答案或预言者暴露作为替代解释，它不是论文的组织假设。

所得图像在风格上比自我进化工作或自我进化仅利用评估者更严谨。修订后的工件可以导致真正的行为改进，但展示的效果往往是狭隘的。单次运行或选定的收益不必在新的部署中存活。有用的traces和feedback必须转化为有效的编辑，被采纳到持久的工件中，并保持行为上的一致性。最后，更高的分数可以反映与测量表面的兼容性，而不是更广泛任务行为。

我们的核心贡献是一种基于主张的生命周期审计：每个报告的收益都与支持它的干预措施以及干预措施允许的最强结论相匹配。我们做出了三项贡献：

- **识别框架**。我们将报告的收益与保持、行为范围、进化与选择途径以及测量依赖性分开。
- **生命周期审计**。我们在三种具有代表性的技能进化机制上应用匹配干预措施，而不会将异构研究变成方法排行榜。
- 报告规范。我们明确从选定收益到有限范围迁移的各层主张所需的演化来源、对照和支撑材料。

2. 问题公式化

2.1. 技能修订与评估

在修订轮次 r 中，优化器更新当前持久的技能工件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 s_r 是当前技能， τ_r 是执行轨迹，而 f_r 是可能包含验证器结果、黄金答案、诊断或评估器反馈的反馈。运算符 O 表示方法完整的更新路径，包括其原生合并、验证、选择或采用机制。一个被拒绝或从未被采用的生成候选不是 s_{r+1}。

代理随后使用冻结的技能执行任务 x：

$$o_r = A(x, s_r, z_r), \tag{2}$$

z_r 表示模型采样、工具使用和环境状态，而 o_r 是结果输出或人工状态变化。一个验证器将输出映射到分数：

$$y_r = V(x, o_r). \tag{3}$$

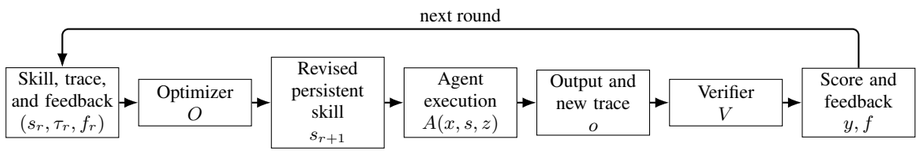


Fig. 1. Minimal self-evolution lifecycle. The update operator includes any method-native validation, selection, merge, or adoption before the revised skill is retained.

2.2. The Identification Question

Most evaluations observe:

$$y_{r+1} > y_r. \tag{4}$$

This inequality describes two evaluations; it does not identify what the persistent skill learned. The revised artifact may encode a retained behavioral improvement, the selected score may depend on optimizer or execution variation, or the output may better match the verifier's measurement surface. Infrastructure-invalid observations, such as a verifier that never reaches its semantic assertions, are neither successes nor failure mechanisms and must be repaired or excluded.

Accordingly, we separate two questions:

1. **Effect existence and scope:** Does the revised skill retain a behavioral improvement under fresh deployment, and how far does that effect extend under controlled task variation?
2. **Effect attribution:** Which feedback, revision, selection, artifact, execution, and measurement mechanisms produced the observed score change?

The following experiments address these questions in turn.

3. DIAGNOSING THE SCOPE OF SKILL UTILITY

The first diagnostic question is where a completed skill is useful. A positive source-task result can arise from content that applies across instances or from a procedure fitted to the original task. We distinguish these possibilities by first examining paired utility in two standard-task runs, then deploying frozen skills under three forms of task variation. Inspection of skill content and execution traces connects the resulting utility profiles to decisions made during deployment.

3.1. Experimental Setup

We evaluate SkillEvolver [5], SPARK [6], and CoEvoSkills [7] on SkillsBench [8], with two runs per method. Each run pairs an execution using the generated or evolved skill with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

We also count tasks with positive, negative, and zero differences.

3.2. Source-Task Gains Recur across Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. Their benefit is therefore more consistent across runs than their absolute skill-condition scores suggest.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. These results establish recurring source-task utility. We next ask whether the same frozen skills remain useful after the task changes.

3.3. Task Changes Separate Different Scopes of Utility

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. These variations test whether a frozen skill remains useful when the executor must reinterpret inputs, recompute a solution, or accommodate new requirements.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. Thus, the source-task gains of the latter two methods do not translate into a consistent advantage on changed tasks.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 ($+14.56$). P2 is also where SPARK and CoEvoSkills decline most (-5.23 and -9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. SkillEvolver's positive results across P1–P3 indicate a broader observed scope of skill utility. CoEvoSkills' negative averages across all three changes show that its source-task benefit is particularly sensitive to reuse conditions.

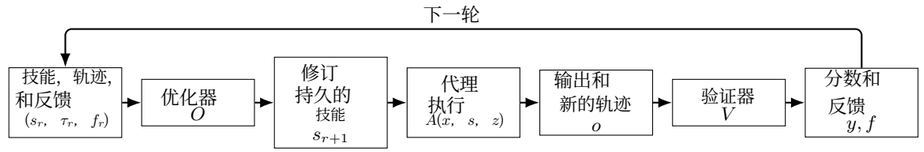


图1。最小化自我进化生命周期。更新算子包含修订技能保留之前的任何方法原生验证、选择、合并或采纳。

2.2. 识别问题

大多数评估观察到：

$$y_{r+1} > y_r. \tag{4}$$

这个不等式描述了两种评估；它并没有指明持久技能学到了什么。修订后的工件可能编码了一种保留的行为改进，选定的分数可能取决于优化器或执行变异，或者输出可能更好地匹配验证器的测量表面。基础设施无效的观察结果，例如从未达到其语义断言的验证器，既不是成功也不是失败机制，必须被修复或排除。

因此，我们将问题分为两个：

1. **效果存在和范围：**修订后的技能在全新部署下是否保留行为改进，以及这种效果在受控任务变异下会延伸多远？
2. **效果归因：**哪些反馈、修订、选择、工件、执行和测量机制导致了观察到的分数变化？

下文实验依次回答这些问题。

3. 诊断技能效用范围

第一个诊断问题是：一个完成的技能在何处有用。正面的源任务结果可能来自适用于多个实例的内容，也可能来自针对原始任务拟合的程序。我们通过首先检查两个标准任务运行中的配对效用，然后以三种任务变化形式部署冻结技能，来区分这些可能性。检查技能内容和执行轨迹，可以将产生的效用曲线与部署过程中做出的决策联系起来。

3.1. 实验设置

我们在技能基准 [8]，上评估 SkillEvolver [5]、SPARK [6]，和协同进化技能 [7]，每种方法运行两次。每次运行将使用生成或进化的技能的执行与相同任务的无技能执行配对。每种方法的两次运行都使用相同的任务集：SkillEvolver 使用 81 个任务，SPARK 使用 84 个任务，协同进化技能使用 85 个任务。

我们报告 0-100 分量级的平均验证器奖励，以及技能和无技能条件下的平均任务内差异。对于包含 N 个任务的运行，配对增益为

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

我们还统计了具有正差异、负差异和零差异的任务。

3.2. 源任务收益跨运行循环

所有三种方法在两次运行中都具有正的平均配对增益（表1）。SkillEvolver获得了25.61和25.02分，CoEvoSkills获得了10.24和9.76分，SPARK获得了1.84和6.45分。尽管SkillEvolver和CoEvoSkills的无技能分数发生了巨大变化，但它们仍然保持了相似的增益。因此，它们在不同运行中的收益比绝对技能条件分数所暗示的更为一致。

任务级别的结果也存在差异。SkillEvolver 在每次运行中至少有四倍的改进而非退化；CoEvoSkills 在每次运行中改进 17 个任务，伴随九次和十次退化。SPARK 的首次运行几乎平衡，而第二次运行有 13 次改进和五次退化。这些结果确立了循环的源任务效用。我们接下来询问，在任务变化后，相同的冻结技能是否仍然有用。

3.3. 任务变化分离不同效用范围

我们将每个源任务技能应用于无进一步进化的受控变体。输入表示变化（P1）会修改名称、列排列或模式；任务实例变化（P2）会替换数据或实例结构；耦合需求变化（P3）会修改或添加必须一起满足的要求。这些变体测试冻结技能在执行者必须重新解释输入、重新计算解决方案或适应新要求时是否仍然有用。

完整记录包含 222 个变体和 886 个条件-等级结果，涵盖无技能和三种方法。表 2 总结了 222 个 SkillEvolver 变体的历史和恢复评估，以及 SPARK 和 CoEvoSkills 各 221 个，将每种技能条件与同一变体上的无技能结果配对。

源任务上的提升并不意味着有效泛化。三种方法在两次源任务运行中的平均奖励均有提升，但变体结果出现分化。SkillEvolver 的平均增益为 11.94 分；SPARK 接近零，为 +0.65 分；CoEvoSkills 则低于无技能基线，为 −5.01 分。SkillEvolver 改善的变体多于退化的变体（47 对 20），而 SPARK 和 CoEvoSkills 相反。因此，后两种方法在源任务上的收益并未转化为变化任务上的一致优势。

替换实例会产生最大的差异。在所有三种变化下，SkillEvolver 均保持正值，其在 P2 (+14.56) 上获得最大增益。P2 也是 SPARK 和协同进化技能下降最严重的位置 (−5.23 和 −9.51)。SPARK 在表示和需求变化下仍保持正值，但在底层实例变化时失去优势。SkillEvolver 在 P1–P3 上的积极结果表明其观察到的技能效用范围更广。协同进化技能在所有三种变化下的负面平均值表明，其源任务收益对重用条件特别敏感。

Method	Run	Tasks	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

Table 1. Two runs per method on SkillsBench. Scores are mean verifier rewards multiplied by 100; Δ is the mean paired gain in percentage points. Each method uses an identical task set across its two runs.

A. Overall utility on task variants							
Method	**Variants**	**No skill**	**Skill**	**Δ**	**Improved**	**Regressed**	**Tied**
SkillEvolver	222	41.87	53.81	+11.94	47	20	155
SPARK	221	41.61	42.25	+0.65	32	37	152
CoEvoSkills	221	41.61	36.60	−5.01	26	41	154

B. Utility by type of task change						
Method	**P1: Representation**		**P2: Instance**		**P3: Requirements**	
	N	Δ	*N*	Δ	*N*	Δ
SkillEvolver	77	+11.85	68	+14.56	77	+9.72
SPARK	77	+2.00	68	−5.23	76	+4.53
CoEvoSkills	77	−3.59	68	−9.51	76	−2.42

Table 2. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Panel A reports overall outcomes; Panel B separates representation, instance, and requirement changes. Each method is paired with its own same-variant no-skill baseline. Scores and within-variant differences use the scale in Table 1.

3.4. What the Evolved Skills Preserve

Successful skills retain guidance for choosing and checking. In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. The SkillEvolver skill explicitly organizes candidate discovery, signal diagnosis, and refinement, including checks for misleading peaks. This case suggests that preserving criteria for accepting a result can be more useful under task changes than retaining a search routine alone. Both skills fail the replacement-instance P2 variant, so the successful checks address only part of this task family's variation.

A similar distinction appears in scheduling. The SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using the CoEvoSkills skill yields an otherwise valid schedule that fails the requirement to place each operation at its earliest feasible time. The SkillEvolver skill explicitly includes this local-minimality requirement and baseline-relative repair rules. Here, successful reuse depends on preserving the task's repair conditions in the skill.

The feedback used to build a skill helps explain its contents. The two methods differ in how they turn experience into revisions. SkillEvolver compares successful and failed deployments of a candidate skill, then patches guidance that was missing or misleading. Its independent audit also checks for reliance on training-instance constants [5]. These mechanisms directly target both omitted conditions and instance-specific assumptions. The rules retained in our scheduling and signal-detection skills are consistent with that emphasis: they describe what to check when reusing a procedure.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier; the ground-truth oracle returns only an opaque pass/fail signal [7]. Consequently, discovering an omitted condition depends on the surrogate reconstructing it from the task and observed outputs. This offers a plausible explanation for a skill that covers many feasibility checks but misses a condition needed for successful reuse, as in the scheduling example. Both methods generate reusable skill packages; the relevant difference is which failure information becomes persistent guidance. Our interpretation is that feedback coverage and abstraction from the source instance help determine the scope of the resulting skill.

Connecting utility to the evolution process. The results identify useful skills and differences in their applicability; the examples suggest how retained guidance relates to those differences. The next studies test where improvements enter the lifecycle, using parent-to-final comparisons and interventions on feedback, selection, skill content, and verification in SkillRevise, SkillHone, and SkillOpt.

4. IDENTIFYING RETAINED SKILL EFFECTS

4.1. Evaluation Design and Contrasts

We analyze three persistent-skill systems with distinct revision mechanisms. SkillRevise converts execution traces and diagnoses into bounded repair principles [2]. SkillHone accumulates practice history and iteratively reflects on the persistent skill [1]. SkillOpt produces structured bounded edits and uses a validation gate and rejected buffer before adoption [3]. The systems are studied as representative mechanisms, not as directly comparable leaderboard entries.

方法	Run	任务	无技能	技能	Δ	改进	退化	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
协同进化技能	1	85	42.21	52.44	+10.24	17	9	59
协同进化技能	2	85	23.54	33.31	+9.76	17	10	58

表1。每个方法在技能基准上运行两次。分数是平均验证奖励乘以100； Δ 是平均配对增益的百分点。每种方法在其两次运行中使用相同的任务集。

A. 任务变体上的整体效用							
方法	**变体**	**No skill**	**技能**	**Δ**	**改进**	**退化**	**Tied**
SkillEvolver	222	41.87	53.81	+11.94	47	20	155
SPARK	221	41.61	42.25	+0.65	32	37	152
协同进化技能	221	41.61	36.60	−5.01	26	41	154

B. 按任务变更类型划分的效用						
方法	**P1：表示**		**P2：实例**		**P3：需求**	
	N	Δ	*N*	Δ	*N*	Δ
SkillEvolver	77	+11.85	68	+14.56	77	+9.72
SPARK	77	+2.00	68	−5.23	76	+4.53
协同进化技能	77	−3.59	68	−9.51	76	−2.42

表 2. 任务变化下的技能效用：历史评估与恢复评估的描述性汇总。面板 A 报告总体结果；面板 B 区分表示、实例和需求变化。每种方法均与相同变体上的无技能基线配对。得分及变体内差值采用表 1 的尺度。

3.4. EvolvedSkills 保留区是什么?

成功的技能保留选择和检查的指导。 在系外行星探测中，SkillEvolver 技能在输入列重新排列（P1）和任务需要两个信号（P3）后支持成功探测。协同进化技能在这两种变体中均失败：其使用导致恒星调制被报告为目标信号，而一个信号及其谐波被视为两个独立信号。SkillEvolver 技能明确组织候选发现、信号诊断和细化，包括对误导性峰值的检查。此案例表明，在任务变化下保留接受结果的准则可能比保留搜索程序本身更有用。两种技能均失败于替换实例 P2 变体，因此成功的检查仅解决了该任务家族变化的一部分。

协同进化技能在调度中也有类似的区别。SkillEvolver 和 SPARK 技能都在三种变体上都成功；而协同进化技能仅在 P3 上成功。在 P1 中，使用协同进化技能会产生一个否则有效的调度，但会失败于要求每个操作都放置在其最早可行时间的要求。SkillEvolver 技能明确地包括了这一局部最优性要求和基线相对的修复规则。在这里，成功的重用取决于在技能中保留任务的修复条件。

用于构建技能的反馈有助于解释其内容。 这两种方法在将经验转化为修订方面有所不同。SkillEvolver 比较候选技能的成功和失败部署，然后修补缺失或误导性的指导。其独立的审计还会检查对训练实例常量的依赖 [5]。这些机制直接针对遗漏的条件和实例特定的假设。我们调度和信号检测技能中保留的规则与这种侧重一致：它们描述在使用过程时需要检查的内容。

协同进化技能从协同进化的代理验证器处获取详细的修订反馈；而真实情况神谕器仅返回一个不透明的通过/失败信号 [7]。因此，发现遗漏的条件取决于代理从任务和观察到的输出中重建它。这为一种涵盖许多可行性检查但遗漏了成功重用所需条件的技能提供了一个合理的解释，就像调度示例中那样。这两种方法都生成可重用的技能包；相关的区别在于哪些失败信息成为持久的指导。我们的理解是，反馈覆盖范围和从源实例的抽象有助于确定结果技能的范围。

将效用连接到进化过程。 结果识别了有用的技能及其适用性差异；示例表明保留的指导与这些差异的关系。下一步研究测试改进进入生命周期的位置，使用亲本到最终的比较以及针对反馈、选择、技能内容和验证在 SkillRevise、SkillHone 和 SkillOpt 中的干预。

4. 识别保留技能效果

4.1. 评估设计与对比

我们分析了三种具有不同修订机制的持久技能系统。SkillRevise将执行轨迹和诊断转换为有界的修复原则 [2]。SkillHone积累练习历史并迭代地反思持久技能 [1]。SkillOpt生成结构化的有界编辑，并在采用前使用验证门和拒绝缓冲区 [3]。这些系统被作为代表性机制进行研究，而不是直接可比的排行榜条目。

Panel and contrast	Main result	Supported inference and boundary
Panel A: Retention under fresh deployment		
Historical pass → fresh repeats	12/24 tasks flip within at least one frozen condition; selected-outcome retention is 47.1% for SkillRevise and 55.0% for SkillHone	A selected pass is not a deployment probability; denominators differ and are not a method ranking.
SkillRevise final → no-skill	16/48 vs. 14/48; +4.2 pp, 95% CI $[-10.4, 20.8]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
SkillHone final → no-skill	12/48 vs. 14/48; −4.2 pp, 95% CI $[-20.8, 14.6]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
Panel B: Revision effects under controlled variations		
Invoice parent → final	$0/12 \rightarrow 12/12$; three new same-contract corpora: $0/9 \rightarrow 9/9$	Stable but narrow same-contract transfer.
Energy parent → final	Original task plus 5% parameter variant: $1/6 \rightarrow 5/6$	Directional parameter transfer in a small stochastic case.
SkillOpt SearchQA parent → final	Independent held-out $9/24 \rightarrow 17/24$; stable-transfer gate fails	Local held-out gain, not stable general procedure learning.

Table 3. Evidence for retained skill effects. Panel A contains 24 targeted tasks, three conditions, and two fresh deployments per condition (144/144 behavior-valid runs). Different historical-retention denominators preclude horizontal method ranking. The no-skill condition calibrates execution and selection variability; final-versus-no-skill does not identify revision effects.

At the paper level, the evidence is exploratory: individual interventions use frozen inputs and predeclared gates, but the combined studies are designed to identify candidate mechanisms rather than estimate benchmark-wide prevalence. A targeted 24-task SkillsBench panel tests whether historical outcomes survive fresh deployment. The panel is stratified to expose stability failures and method disagreements; it is not a random sample. Three controlled mechanism cases then compare complete parent and final artifacts under predeclared variations. Within each contrast, we hold the executor model, harness, task instance, budget, and verifier version fixed unless one is the named intervention; exact prompts, hashes, seeds, and stopping decisions are preserved in the artifact package. The task is the primary analysis unit, while repeated runs are deployments of the same task-condition pair.

Four comparisons must remain distinct. Historical-pass to fresh-repeat comparisons test retention of a selected outcome, not advantage over a baseline. Final-to-no-skill comparisons estimate fresh utility, not the effect of revision. Only actual-parent to final comparisons identify a revision effect. Applying that comparison to controlled variants tests bounded transfer, not population-average benefit.

4.2. Retention under Fresh Deployment

This experiment tests whether historically selected outcomes survive when the final artifact is frozen and redeployed. Holding task, skill, and condition fixed does not make execution deterministic. Across the 24-task panel, 12 tasks change success status between their two repeats in at least one condition. Aggregate fresh success is 14/48 without a skill, 16/48 with the SkillRevise final artifact, and 12/48 with the SkillHone final artifact. Both matched-lift intervals cross zero (Table 3).

These results establish an identification failure in the common reporting practice: cached or selected success cannot be read as stable deployment capability. They do not establish that either revision method is ineffective. The panel is purposive, and it lacks each method's actual parent condition. Its proper conclusion is narrower: on this sample, neither final-versus-no-skill comparison establishes a clear average fresh-deployment advantage, and fresh repeats are necessary before a selected score is called retained.

4.3. Revision Effects under Controlled Variations

These mechanism probes test whether the actual parent-to-final revision effect persists under predeclared task changes. In Invoice, the complete final skill repairs all 12 matched deployments and retains the effect on nine deployments drawn from three disjoint corpora that preserve the contract while changing the documents. This is strong evidence of a stable, same-contract revision effect, but not of broad cross-contract generalization.

面板与对比	主要结果	支持推理与边界
面板 A：新部署下的保留率		
历史通过 → 新重复	12/24 任务至少翻转一种冻结条件；选择结果保留是 47.1% 对于 SkillRevise 和 55.0% 对于 SkillHone	一个选择通过不是部署概率；分母不同，并且不是方法排名。
技能修订最终 → 无技能	16/48 vs. 14/48； +4.2 pp，95% CI $[-10.4, 20.8]$	在此目标面板上没有明显的全新效用优势；不是从父级到最终修订的估计。
技能精炼最终 →无技能	12/48 与 14/48； −4.2 pp，95% CI $[-20.8, 14.6]$	在此目标面板上，没有明显的全新效用优势；不是从父版到最终修订的估计。
面板 B：受控变化下的修订效果		
发票父版 →最终	$0/12 \rightarrow 12/12$; 三个新的相同合同语料库: $0/9 \rightarrow 9/9$	稳定但狭窄的相同合同迁移。
能量父 →最终	原始任务加5%参数变体: $1/6 \rightarrow 5/6$	小规模随机情况下的方向参数迁移。
SkillOpt SearchQA 父 → 最终	独立保留集 $9/24 \rightarrow 17/24$; 稳定迁移门控失败	本地保留集增益，非稳定泛化程序学习。

表3。保留技能效应的证据。面板A包含24个目标任务、三个条件，每个条件有两次新鲜部署（144/144行为有效运行）。不同的历史保留分母排除了水平方法排名。无技能条件校准执行和选择变异性；最终与无技能对比不能识别修订效应。

在论文层面，证据是探索性的：单个干预使用冻结的输入和预声明的门，但综合研究旨在识别候选机制，而不是估计基准范围内的普遍性。一个有针对性的24任务SkillsBench面板测试历史结果是否能在全新部署中存活。该面板按比例分层以暴露稳定性失败和方法分歧；它不是一个随机样本。然后三个受控机制案例在预声明的变化下比较完整的父级和最终工件。在每个对比中，我们保持执行器模型、测试框架、任务实例、预算和验证器版本固定，除非它是命名的干预；精确的提示、哈希值、种子和停止决策保留在工件包中。任务是主要分析单元，而重复运行是相同任务条件对的部署。

必须保持四个比较的独立性。历史-新鲜重复比较测试的是选定结果的保持，而非相对于基线的优势。最终-无技能比较估算的是新鲜效用，而非修订的影响。只有实际父代到最终的比较才能识别修订效应。将此比较应用于控制变量测试的是有限迁移，而非群体平均收益。

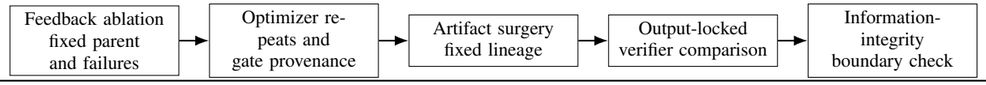
4.2. 新部署下的保持能力

本实验测试的是当最终产品冻结并重新部署时，历史选定的结果是否仍然有效。固定任务、技能和条件并不能使执行结果确定。在24任务面板中，12个任务在至少一个条件下两次重复的成功状态发生了变化。没有技能时，新鲜成功率为14/48；使用SkillRevise最终产品时，新鲜成功率为16/48；使用SkillHone最终产品时，新鲜成功率为12/48。两个匹配提升间隔都穿过零点（表3）。

这些结果确立了一种在常见报告实践中存在的识别失败：缓存的或选定的成功不能被解读为稳定的部署能力。它们并没有确立任何一种修订方法都是无效的。该小组是有目的的，并且缺乏每种方法的实际父条件。它的正确结论范围更窄：在这个样本中，无论是最终与无技能的比较，都没有确立明显的平均新鲜部署优势，在称为保留选定分数之前，新鲜重复是必要的。

4.3. 控制变化下的修订效果

这些机制探针测试在实际的父节点到最终修订效果在预定义的任务变更下是否仍然存在。在发票中，完整的最终技能修复了所有12个匹配的部署，并保留了从三个保持合同但更改文档的独立语料库中抽取的九个部署的效果。这是稳定、相同合同修订效果的强有力证据，但不是广泛跨合同泛化的证据。



Lifecycle stage	SkillRevise	SkillHone	SkillOpt
Feedback and revision	Diagnosis ablation; revision traces	Feedback-view ablation	Feedback branches; optimizer repeats
Selection and adoption	Revision history	Iterative honing history	Native validation gate; accepted and rejected candidates
Artifact mediation	Not yet matched at the same strength	Invoice and Energy surgery	SearchQA component surgery
Measurement	No output-locked comparison yet	Radar verifier swap	Spreadsheet dual scoring
Information integrity	Artifact forensic	Artifact forensic	Wrong-metadata and oracle controls

Fig. 2. Lifecycle attribution design and current evidence coverage. The upper panel locates each intervention; the lower panel records method coverage. Populated cells do not imply equivalent strength, and missing cells bound cross-method claims.

The Energy case provides weaker but directionally consistent evidence: the complete final artifact improves the original setting and a 5% parameter variant in five of six deployments, compared with one of six for the parent. The sample is too small to support a prevalence or generality claim. In SkillOpt SearchQA, the final artifact improves independent held-out accuracy from 9/24 to 17/24, yet fails the predeclared stability threshold. It therefore supports local held-out benefit rather than stable transferable procedure learning.

Taken together, revised skill artifacts can produce retained behavioral effects, but a selected benchmark pass alone does not identify such an effect, and the effects established so far remain narrow rather than broadly generalizable.

5. ATTRIBUTING OBSERVED SKILL GAINS

5.1. Lifecycle Interventions

The experiments in this section intervene at different lifecycle stages. They do not estimate one common effect size. Each intervention changes one object while holding its relevant lineage fixed: feedback-view studies hold the parent and failures fixed; optimizer-repeat studies hold the input evidence fixed; gate audits preserve accepted and rejected candidates; artifact surgery holds the surrounding artifact fixed; and verifier counterfactuals hold the output fixed.

Figure 2 makes the current coverage and its gaps explicit. The shared audit logic is that feedback access, candidate revision, validation acceptance, adoption into the persistent artifact, and retained behavioral improvement are distinct events.

5.2. Revision and Selection Pathways

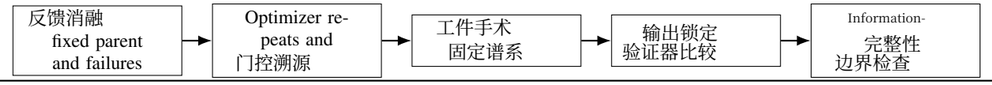
These interventions test whether the observed endpoint depends on the feedback view, optimizer sample, or selection gate. In SkillRevise, the complete three-revision pipeline succeeds on 53/86 tasks, compared with 28/86 when diagnosis is removed. Task-level discordance is 26 gains versus one loss, and 2,326 of 3,648 revision entries explicitly use verifier evidence. This demonstrates that diagnosis changes the selected endpoint and that the revision process is strongly evaluator-facing. It does not, by itself, establish which changes are retained under fresh deployment.

SkillHone separates access to feedback from stable benefit. Detailed sanitized, binary, and redacted-tail feedback obtain first-deployment success on 4/12, 4/12, and 5/12 cases, while four of 18 repeated cells flip. Feedback can trigger revisions, but these data do not support a stable advantage for one feedback view.

SkillOpt further separates candidate production from attribution and adoption. In matched SearchQA optimizer repeats, raw-gold and outcome-only views each produce four applied edits. Yet the preregistered between-condition effect size is only $D = .044 < .20$, despite weak lexical decodability, so a single patch cannot be cleanly attributed to feedback view. Across outcome-only and trajectory-only branches, all 6/6 candidates change and 3/6 pass the gate, but every acceptance is driven by the same Blackwater validation item. In OfficeQA, a selected one-item gain disappears in fresh matched deployment. Thus edit production, gate acceptance, and retained improvement cannot be collapsed into one event.

5.3. Artifact-Level Mediation

This experiment tests whether a feedback-derived component, rather than unrelated edits in the full artifact, causes the behavior after adoption. Let P denote the complete parent, F the complete final artifact, $P + B$ the parent with component B, and $F - B$ the final artifact with that component removed.



生命周期阶段	技能修订	技能精炼	技能优化
反馈与修订	诊断消融；修订轨迹	反馈视图消融	反馈分支；优化器重复
选择与采纳	修订历史	迭代打磨历史	本地验证门；已接受和被拒绝的候选者
文物调解	尚未在相同力量	发票和能源手术	SearchQA组件手术
测量	尚未有输出锁定比较	雷达验证器交换	电子表格双重评分
信息完整性	文物取证	文物取证	错误元数据和预言机控制

图2。生命周期归因设计　　n和当前的证据覆盖范围。上方面板定位每个干预措施；下方面板记录方法　　od
覆盖率。已填充的单元格不　　仅等效强度，且缺失的单元格绑定跨方法声明。

能源案例提供了较弱但方向上一致的证据：最终的完整工件改进了原始设置，并且在六次部署中的五次出现了5%的参数变体，而父代只有六次部署中出现一次。样本太小，无法支持普遍性或一般性声明。在SkillOpt SearchQA中，最终工件将独立保留的准确率从9/24提高到17/24，但未能通过预声明的稳定性阈值。因此，它支持局部保留的好处，而不是稳定的可迁移程序学习。

综合来看，修订后的技能工件可以产生保留的行为效果，但仅通过一个选定的基准测试并不能识别出这种效果，而且迄今为止建立的效果仍然比较狭窄，而不是广泛普适的。

5. 观察到的技能增益归因

5.1. 生命周期干预

本节中的实验在不同的生命周期阶段进行干预。它们不估计一个共同的效果量。每次干预改变一个对象，同时保持其相关谱系固定：反馈视图研究保持父对象和失败固定；优化器重复研究保持输入证据固定；门控审计保留接受的和拒绝的候选；工件手术保持周围工件固定；验证器反事实研究保持输出固定。

图2使当前的覆盖范围及其空白处变得明确。共享的审计逻辑是，反馈访问、候选修订、验证接受、采纳到持久化工件以及保留的行为改进是不同的独立事件。

5.2. 修订与选择路径

这些干预措施测试观察到的终点是否依赖于反馈视图、优化器样本或选择门。在SkillRevise中，完整的三次修订流程在86个任务中有53个成功，而移除诊断时只有28个成功。任务级别的分歧是26个收益对应1个损失，且在3,648个修订条目中有2,326个明确使用验证器证据。这表明诊断改变了选定的终点，且修订过程是强评估者导向的。它本身并不能确定哪些变更在全新部署下被保留。

技能磨炼将访问反馈与稳定收益分离。详细的清理、二进制和红头尾反馈在4/12、4/12和5/12的案例中获得首次部署成功，而18个重复单元中有4个翻转。反馈可以触发修订，但这些数据不支持对某个反馈视图的稳定优势。

技能优化进一步将候选生成与归因和采用分离。在匹配的SearchQA优化器重复中，原始金标和结果仅视图各自产生四个应用编辑。然而，预先注册的组间效果量仅为 $D = .044 < .20$，尽管存在较弱的词汇解码性，因此单个补丁无法清晰地归因于反馈视图。在结果仅和轨迹仅分支中，所有6/6候选都发生变化，3/6通过门控，但每个接受都是由同一个Blackwater验证项驱动。在OfficeQA中，一个选定的单项收益在新鲜匹配部署中消失。因此编辑生成、门控接受和保留的改进不能合并为一个事件。

5.3. 工件级中介

该实验测试的是，是否是源自反馈的组件（而非完整工件中的无关修改）导致了采纳后的行为。令 P 表示完整的父项，F 表示完整的最终工件，$P + B$ 表示包含组件 B 的父项，以及 $F - B$ 表示移除了该组件的最终工件。

Case	Result	Maximum claim
Invoice	$P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$	B is necessary and sufficient for the narrow same-contract effect.
Energy	Local mediation in the original task; necessity and sufficiency do not survive the variant	Local component effect, not transferable mediation.
SearchQA	9 repairs, 0 regressions; stable success 7/12, below the strong gate	Directional mediation on part of the held-out set.

Table 4. Artifact-level mediation evidence.

These interventions establish that an adopted artifact component can cause behavior. They do not show that the same mechanism is prevalent across methods. In particular, SkillRevise lacks a matched surgery of equal strength, and the current cases should be read as existence and boundary evidence.

5.4. Measurement Dependence under Fixed Outputs

This counterfactual tests whether a verdict changes when the output is fixed and only the measurement changes. In the Radar family, all 12 reconstructed outputs reproduce their historical original-verifier outcomes. Of six original passes, only one remains a pass under a reference-calibrated strict verifier; the five valid flips arise from specification-implied checks. The same behavior can therefore receive different valid verdicts because the measurement surfaces differ.

Measurement dependence can also operate in the other direction. For identical Spreadsheet workbook outputs, the reported value-based evaluator scores the complete SkillOpt artifact at 10/12, whereas joint value-and-formula recalculation scores it at 8/12. Representation, cache materialization, or formula inspection can therefore change what the evaluator observes. This dual scoring is not identical in design to the Radar output-locked swap, but it reinforces the narrower point that score is a property of output plus measurement, not behavior alone.

Neither result proves semantic failure, exploitation, or benchmark-wide prevalence. To identify evolution-induced verifier dependence, one would additionally need matched no-skill, parent, and final outputs under both original and calibrated verifiers, and test whether the parent-to-final verifier gap increases.

5.5. Information-Integrity Boundary

This supporting check tests whether restricted answer or oracle information enters and controls the artifact. High-confidence direct-answer or oracle matches are absent from 86 SkillRevise and 85 SkillHone natural artifacts. SkillOpt conditions containing the feedback-derived answer-surface section follow incorrect metadata in 0/15 probes. In contrast, an unsafe hidden-oracle positive control changes probe success from 0/5 to 5/5 and leaves private-oracle or gold markers.

These checks show that the audit can detect direct exposure when it is deliberately introduced, while the natural artifacts do not support direct copying as a universal explanation. They do not prove the absence of implicit answer-following policies or all forms of leakage.

Observed score gains can arise at multiple lifecycle stages: feedback shapes candidate revisions, selection can amplify narrow improvements, adopted artifact components can mediate real behavior, and valid verifiers can disagree on fixed outputs. These mechanisms must be identified separately before a score gain is interpreted as skill learning.

6. FROM REPORTED GAINS TO AUDITABLE CLAIMS

6.1. Evidence Requirements and Claim Boundaries

The audit is not a scalar ladder from *safe* to *unsafe*. Its steps answer independent questions and impose distinct claim boundaries.

Audit step	Required test → maximum supported claim
Validity	Execution and verifier status plus assertion coverage → eligible observation
Retention	Frozen actual-parent/final fresh repeats → retained artifact effect
Behavioral scope	Entity, parameter, input, contract, or family variants → bounded controlled transfer
Evolution pathway	Feedback ablation, optimizer repeats, and gate provenance → pathway dependence or selection mediation
Measurement	Output-locked calibrated verifier comparison → measurement dependence
Information integrity	Provenance, wrong-metadata, and positive control → exploitation or confirmed leakage, when supported

Table 5. Evidence requirements and claim boundaries. The rows answer complementary questions rather than forming a scalar risk ladder.

A selected gain is weaker than a retained artifact effect; a retained effect on one instance is weaker than controlled transfer. Evolution-pathway and measurement claims remain meaningful even when retention is not established. Conversely, verifier-aware language or evaluator dependence must not be relabeled as leakage without restricted-information provenance and behavioral evidence.

6.2. Lifecycle Reporting Protocol

An auditable self-evolution report should preserve the following minimum evidence:

- **Before optimization:** parent artifact, task and split identity, failure pool, environment, and verifier version.
- **During revision:** optimizer-visible traces and feedback, candidate patches, optimizer samples, prompts, and usage.
- **At selection and adoption:** accepted and rejected artifacts, gate items, per-item decisions, and exact adopted artifact.
- **At deployment:** frozen artifact, actual-parent and no-skill conditions, fresh repeats, controlled variants, and output bytes.
- **At scoring:** verifier logs, assertion coverage, reference calibration, and any alternative verifier applied to the same output.
- **For cost:** separate optimizer, executor, verifier, and invalid-attempt ledgers.

Case	结果	最大主张
发票	$P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$	B 是必要的, sufficient for the 窄相同合约 effect.
能量	本地中介在 原始任务；必要性和 充足性无法生存 变异体	本地组件 非效果，而是 可转移的 中介。
SearchQA	9次修复，0次回归； 稳定成功7/12，低于 强门	方向性中介 在部分 留出集。

表4。工件级中介证据。

这些干预措施表明，采纳的工件组件可以导致行为。它们并未证明相同的机制在所有方法中都普遍存在。特别是 SkillRevise 缺乏同等强度的匹配手术，当前案例应被视为存在性和边界证据。

5.4. 测量依赖性-固定输出

这个反事实测试当输出固定且仅测量变化时，裁决是否会改变。在Radar系列中，所有12个重建输出都重现了它们的历史原始-验证器结果。在六个原始通过中，只有一个在参考校准的严格验证器下仍然是通过；五个有效翻转来自规范隐含的检查。因此，相同的行为可以因为测量表面不同而获得不同的有效裁决。

测量依赖性也可以在相反的方向上起作用。对于相同的电子表格工作簿输出，基于值的评估器将完整的SkillOpt工件评为10/12，而联合值和公式重新计算将其评为8/12。因此，表示、缓存材料化或公式检查可以改变评估器观察到的内容。这种双重评分在设计中并不与Radar输出锁定交换相同，但它加强了更狭窄的观点，即分数是输出加测量的属性，而不仅仅是行为。

Neither结果证明语义失败、利用或基准范围内的普遍存在。为了识别进化引起的验证器依赖，还需要在原始和校准的验证器下匹配无技能、父代和最终输出，并测试父代到最终验证器差距是否增加。

5.5. 信息完整性边界

此支持性检查测试是否受限答案或预言机信息进入并控制该工件。高置信度的直接答案或预言机匹配在 86 个 SkillRevise 和 85 个 SkillHone 自然工件中不存在。包含反馈衍生答案表面部分的 SkillOpt 条件在 0/15 次探测中遵循错误元数据。相比之下，一个不安全的隐藏预言机正向控制将探测成功率从 0/5 改变为 5/5 并留下私有预言机或黄金标记。

这些检查表明，审计可以在故意引入时检测到直接暴露，而自然产生的伪影并不支持直接复制作为普遍解释。它们并不能证明不存在隐式答案跟随策略或所有形式的泄漏。

观察到的分数增益可能出现在多个生命周期阶段：反馈塑造候选修订，选择可以放大狭窄的改进，采用的伪影组件可以调解实际行为，有效的验证器可以对固定输出产生分歧。在将分数增益解释为技能学习之前，必须单独识别这些机制。

6. 从报告收益到可审核的声明

6.1. 证据要求与主张边界

审计并非从 安全 到 不安全 的标量阶梯。它的步骤回答独立问题，并施加不同的主张边界。

审计步骤	必需测试 →最大支持的声明
有效性	执行和验证器状态加上断言 覆盖率 → 符合条件的观察
保留	冻结实际父/最终新鲜重复 → 保留 工件效果
行为 范围	实体、参数、输入、合约或家族 变体 → 有界控制转移
进化 路径	反馈消融、优化器重复和门控 溯源 →路径依赖或选择 调节
测量输出锁定	校准验证器比较 → 测量依赖性
信息 完整性	来源、错误元数据和阳性对照 → 利用或确认泄露，当 受支持

表5。证据要求和声明边界。The rows answer complementary questions，而不是形成一个标量风险梯子。

一个选定的收益比保留的工件效应弱；一个实例上的保留效应比受控迁移弱。即使保留没有建立，进化路径和测量声明仍然有意义。相反，验证器感知语言或评估器依赖必须在有限信息的来源和行为证据下才能不被重新标记为泄漏。

6.2. 生命周期报告协议

一个可审计的自进化报告应该保留以下最低证据：

- **优化前：** 父工件、任务和分割身份、失败池、环境和验证器版本。
- **在修订期间：** 优化器可见的轨迹和反馈、候选补丁、优化器样本、提示和用法。
- **在选型和采纳时：** 已接受和拒绝的工件、门控项、单项决策，以及精确采纳的工件。
- **在部署时：** 冻结的工件、实际父项和无技能条件、新鲜重复、受控变体，以及输出字节。
- **在评分时：** 验证器日志、断言覆盖率、参考校准，以及应用于相同输出的任何替代验证器。
- **对于成本：** 分离优化器、执行器、验证器和无效尝试账本。

The complete artifact appendix should additionally record hashes, commands, seeds, environment revisions, timeouts, and stopping decisions. This protocol turns a post-evolution score into an auditable lineage and limits the paper's language to the strongest matched contrast actually performed.

7. RELATED WORK

7.1. Skill Evolution and Empirical Evaluation

Agent-skill research spans generation, revision, selection, retrieval, organization, and reinforcement learning. SkillsBench and SWE-Skills-Bench compare skill and no-skill utility across diverse or software-engineering tasks [8, 9]. SkillLearnBench evaluates continual skill generation [10], while SkillRevise, SkillHone, SkillOpt, SkillRL, and CoEvoSkills propose different revision or learning loops [1–4, 7]. Recent empirical work broadens the object of study: From Raw Experience to Skill Consumption separates experience generation, extraction, and consumption [11]; How Well Do Agentic Skills Work in the Wild studies realistic skill retrieval [12]; Skill-Coach evaluates process-level skill use [13]; and SkillJuror isolates the effect of skill organization [14].

These studies show that outcome-only evaluation is already recognized as incomplete. Our focus is narrower and complementary: when a persistent skill is revised and the post-revision score rises, what evidence identifies a retained artifact effect rather than a selected or measurement-dependent endpoint? We therefore organize evidence by lifecycle contrast rather than by method score.

7.2. Evaluation and Attribution of Agent Improvement

Agent evaluation increasingly examines process quality, sequential adaptation, verifier reliability, and counterfactual behavior. Counterfactual Trace Auditing measures how a provided skill changes an agent trajectory [15]. SEA-Eval evaluates self-evolving agents beyond isolated episodes [16]. Co-evolving verifier work and grader audits study interactions between the policy and its measurement signal [7, 17]. More broadly, reward hacking and test-set contamination research demonstrates that performance on a proxy or exposed test need not imply the intended capability [18, 19].

We combine these concerns within the lifecycle of persistent skill revision. The central distinction is between retention, behavioral scope, evolution-and-selection pathway, and output-level measurement dependence. Information-integrity checks play a supporting role: they bound direct answer or oracle exposure, but do not convert every verifier-aware adaptation into a leakage claim.

8. LIMITATIONS

Our evidence is mechanism-focused rather than a benchmark-wide prevalence study. The 24-task retention panel is purposive and contains only two fresh deployments per condition; it exposes instability but cannot estimate deployment rates. The controlled Invoice, Energy, SearchQA, Radar, Spreadsheet, and OfficeQA probes use small, heterogeneous task families rather than a fully crossed method-by-intervention design. SkillRevise, for example, lacks matched artifact surgery and output-locked measurement evidence of the same strength, while the strongest verifier swap is concentrated in the Radar family. We therefore do not claim that any observed mechanism is universal or that one method outperforms another.

Several conclusions are also bounded by observability. Output-locked comparisons identify measurement dependence but do not determine which verifier best captures intended behavior without an external specification. Artifact forensics and wrong-metadata controls can rule out simple direct exposure, but not implicit answer-following policies or every form of contamination. Finally, our scope is frozen-agent, feedback-driven systems with persistent external artifacts; parameter-updating self-evolution, multimodal skills, and online environments may exhibit different lifecycle dynamics. Confirmatory evidence will require a shared sampling plan that fully crosses actual parent and final artifacts, fresh repeats, controlled variants, and calibrated verifiers.

9. CONCLUSION

We diagnose where the gains from self-evolving persistent skills reside and how far they extend. All three methods in the standard-task evaluations have positive paired gains in both runs, but task variation separates their utility profiles. SkillEvolver remains positive across representation, instance, and requirement changes; SPARK combines gains on two probes with a decline on instance changes; CoEvoSkills declines on all three. The inspected traces identify decision criteria and correctness checks expressed during successful reuse. Complementary lifecycle interventions show that adopted artifact components can change behavior, selected outcomes can disappear under fresh deployment, and fixed outputs can receive different verdicts under different verifiers. Together, these findings distinguish useful artifacts, their behavioral scope, and the processes that produce or measure their apparent improvement.

A higher benchmark score should therefore be reported only at the strongest level supported by matched evidence: a selected gain, a retained artifact effect, bounded controlled transfer, evolution-pathway dependence, or measurement dependence. We provide a lifecycle audit that links each claim to the required intervention and reporting evidence. Claims of transferable skill learning require frozen parent-to-final improvement that survives fresh deployment and predeclared task variation under valid measurement; weaker evidence should be reported more narrowly.

10. REFERENCES

[1] Zhiwei Li and Yong Hu, "SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History," 2026.

[2] Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyan Ji, Yauwai Yim, Hongyu Luo, Xiyu Ren, Chenyu Ruan, Haoran Li, and Yangqiu Song, "SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision," 2026.

[3] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo, "SkillOpt: Executive Strategy for Self-Evolving Agent Skills," 2026.

[4] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao, "SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning," 2026.

完整的工件附录还应记录哈希值、命令、种子、环境修订、超时和停止决策。此协议将进化后的分数转化为可审计的谱系，并限制了论文的语言仅限于实际执行的最强匹配对比。

7. 相关工作

7.1. 技能进化和实证评估

代理-技能研究涵盖生成、修订、选择、检索、组织以及强化学习。技能基准和SWE-技能基准比较技能和无技能在不同或软件工程任务中的效用 [8, 9]。SkillLearnBench评估持续技能生成 [10], , 而SkillRevise、SkillHone、SkillOpt、SkillRL和协同进化技能则提出不同的修订或学习循环 [1–4, 7]。最近的实证工作扩展了研究对象：从原始经验到技能消费将经验生成、提取和消费分开 [11]；代理技能在现实中的表现如何研究现实技能检索 [12]；Skill-Coach评估过程级的技能使用 [13]；而SkillJuror则隔离技能组织的影响 [14]。

这些研究表明，仅评估结果的评价已被认识到是不完整的。我们的关注点更狭窄且互补：当持久的技能被修订且修订后的分数上升时，哪些证据能识别出保留的工件效应而不是被选择或测量依赖的终点？因此，我们按生命周期对比而非按方法分数组织证据。

7.2. 代理改进的评估与归因

代理评估越来越多地关注过程质量、顺序适应、验证器可靠性和反事实行为。反事实轨迹审计衡量一个提供的技能如何改变代理轨迹 [15]。SEA-Eval 评估自进化代理超越孤立事件 [16]。协同进化验证器工作和评分器审计研究策略与其测量信号之间的交互 [7, 17]。更广泛地说，奖励破解和测试集污染研究证明，在代理或暴露的测试上的表现不一定意味着预期的能力 [18, 19]。

我们将这些关注点结合在持久技能修订的生命周期中。核心区别在于保留、行为范围、进化与选择路径以及输出级测量依赖性。信息完整性检查起辅助作用：它们限制直接答案或预言机暴露，但不会将所有验证器感知的适应转换为泄漏声明。

8. 局限性

我们的证据是机制导向的，而不是全范围的基准研究。24任务保持面板是经过精心挑选的，每个条件下只包含两次新鲜部署；它暴露了不稳定性，但无法估计部署率。受控的Invoice、Energy、SearchQA、Radar、Spreadsheet和OfficeQA探针使用小型异构任务家族，而不是完全交叉的方法-干预设计。例如，SkillRevise缺乏匹配工件手术和输出锁定测量的同等强度证据，而最强的验证器交换集中在Radar家族。因此，我们不声称任何观察到的机制是普遍的，也不声称一种方法优于另一种方法。

一些结论也受限于可观察性。输出锁定比较能识别测量依赖性，但在没有外部规范的情况下，无法确定哪个验证器最能捕捉预期行为。文物取证和错误元数据控制可以排除简单的直接暴露，但无法排除隐式答案遵循策略或任何形式的污染。最后，我们的范围是冻结代理、反馈驱动且具有持久外部文物的系统；参数更新的自进化、多模态技能和在线环境可能表现出不同的生命周期动态。确认证据将需要一个完全跨越实际父代和最终文物的共享采样计划、新鲜重复、受控变体和校准验证器。

9. 结论

我们诊断自进化持久技能的收益所在及其影响范围。在标准任务评估中的三种方法在两次运行中都具有正向配对收益，但任务变化区分了它们的效用特征。SkillEvolver 在表示、实例和需求变化中保持正向；SPARK 在两个探针上结合收益，但在实例变化中下降；CoEvoSkills 在所有三个上都下降。检查的轨迹识别了在成功重用时表达的决策标准和正确性检查。互补的生命周期干预表明，采用的组件可以改变行为，选定的结果可能在新的部署下消失，固定的输出在不同验证器下可能收到不同的裁决。这些发现共同区分了有用组件、其行为范围以及产生或测量其明显改进的过程。

因此，只有在有匹配证据支持的最强级别下，才应报告更高的基准分数：一个选定的收益、一个保留的文物效应、有界的受控迁移、进化路径依赖性或测量依赖性。我们提供生命周期审计，将每个主张与所需的干预措施和报告证据联系起来。可迁移技能学习的主张要求冻结的从父级到最终改进，该改进在有效测量下能够经受新鲜部署和预声明的任务变化；较弱的证据应更严格地报告。

10. 参考文献

[1] 李智伟和胡勇，‘SkillHone：通过持久决策历史持续进化代理技能的 harness’，2026.[2] 刘宇轩、苏兆辰、谢凌云、张宇豪、宗庆、郭嘉禾、谢中伟、姬一岩、Yauwai Yim、罗红宇、任希宇、阮晨宇、李浩然和宋阳秋，‘SkillRevise：通过轨迹条件技能修订改进 LLM 编写的代理技能’，2026.[3] 杨一帆、宫子阳、黄伟权、杨启豪、周子伟、黄子睿、李岩、高雪梅、戴琪、刘北、邱凯、杨宇清、陈东东、杨雪、罗崇，‘SkillOpt：自进化代理技能的执行策略’，2026.[4] 夏鹏、陈建文、王汉阳、刘佳琪、曾开德、王宇、韩思伟、周奕阳、赵旭江、陈海峰、郑泽宇、谢驰航和姚华秀，‘SkillRL：通过递归技能增强强化学习进化代理’，2026。

[5] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[6] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[7] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[8] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[9] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[10] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[11] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.

[12] Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang, "How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings," 2026.

[13] Jiayin Zhu, Kelong Mao, Yudong Guo, Dengbo He, Sulong Xu, Simiu Gu, and Yutao Yue, "SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use," 2026.

[14] Zhiyu Chen, Zihan Guo, Bo Huang, Bingwei Lu, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang, "SkillJuror: Measuring How Agent Skill Organization Changes Runtime Behavior," 2026.

[15] Xiaolin Zhou, Jinbo Liu, Li Li, Ryan A. Rossi, and Xiyang Hu, "Counterfactual Trace Auditing of LLM Agent Skills," 2026.

[16] Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Tengfei Wang, Shisong Chen, Jinghao Zhang, Tianjun Pan, Weijia Li, Jiaqing Liang, and Yanghua Xiao, "SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment," 2026.

[17] Xing Zhang, Guanghui Wang, Yanwei Cui, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He, "Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents," 2026.

[18] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané, "Concrete Problems in AI Safety," 2016.

[19] Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto, "Proving Test Set Contamination in Black Box Language Models," 2023.

[5] 张根瑞、朱尔乐、周金峰、贾彩艳和王红宁，‘SkillEvolver：将技能学习作为元技能’，2026.[6] 周阳、董志涵、王志腾、金灿、赵时宇、郭邦伟、顾迪菲、张林军、周慕和Dimitris N. Metaxas，‘证据优于计划：技能蒸馏的在线轨迹验证’，2026.[7]张汉荣、范世成、邹亨利、陈彦凯、王志腾、周嘉宇、李成哲、黄伟哲、姚一飞、郑凯宁、刘雪、李晓晓和Philip S. Yu，‘CoEvoSkills：通过协同进化验证自进化代理技能’，2026.[8] 李祥怡、刘一敏、陈文博、尤冰然、狄宗林、何一峰、郑胜汉、崔炯万、孙建凯、王舒怡、陶初俊、李冰旭、赵宣东、耿鹤佳、吴晓军、周俊伟、陈晓坤、辛汉文、李宇博、曾群红、王迪、王元利、Chaim Roey Ben、蒋鹏浩、沈浩天、孔鲁阳、刘新怡、王润辉、刘宣清、李嘉晨、兰欣、林月倩、叶文高、何俊伟、李松林、张悦、高奕鹏、李一江、马泽、景利强、王天宇、李凯欣、薛一奇、吕浩然、何一卓、田宇晨、吴舒通、王伯伟、高亦璇、陈伯、刘立通、程思凯、鲍嘉俊、童树成、徐淑文、朱舟悦、叶天寒、戚琪、李妙、廖龙泰、谭泽林、石昌、唐锡林、Srinath Tankasala、袁伯勤、钱瑶瑶、涂建红、王晨光、孙一舟、王伟、Aaron Taylor、杨子越、管长坤、董志康、张新宇、Steven Dillmann、李汉中、宋 Dawn，‘SkillsBench：基准测试代理技能在不同任务中的表现’，2026.[9] 韩挺旭、张怡、宋伟、方春荣、陈振宇、孙友成和李丽杰，‘SWE-Skills-Bench：代理技能是否真的有助于现实世界软件工程？’，2026.[10] 钟珊珊、陆怡、宁静洁、万一冰、冯丽涵、敖宇奕、Leonardo F. R. Ribeiro、Dreyer Markus、Sean Ammirati和熊晨艳，‘SkillLearnBench：在现实世界任务上基准测试代理技能生成的持续学习方法’，2026.[11] 黄子睿、徐景文、杨一帆、宫子阳、杨启豪、田木超、王晓华、吕长泽、高雪梅、戴琪、刘北、邱凯、杨雪、陈东东、郑晓庆和罗崇，‘从原始经验到技能消耗：模型生成代理技能的系统研究’，2026.[12] 刘宇健、纪嘉宝、安丽、Tommi Jaakkola、张阳和常时宇，‘代理技能在野外效果如何：基准测试现实场景中 LLM 技能使用’，2026.[13] 朱佳音、毛克龙、郭宇东、何登波、徐索龙、顾思宇和岳宇涛，‘SkillCoach：自进化评估和增强代理技能使用的评分标准’，2026.[14] 陈智宇、郭志涵、黄伯、卢炳伟、林江浩、周元健和张伟楠，‘SkillJuror：测量代理技能组织如何改变运行时行为’，2026.[15] 周晓林、刘金波、李丽、Ryan A. Rossi和胡希阳，‘LLM 代理技能的逆事实轨迹审计’，2026。

[16] 蒋思航, 马立鹏, 洪中华, 王凯毅, 陆志宇, 王腾飞, 陈诗松, 张景浩, 潘天军, 李伟嘉, 梁嘉庆, 肖阳华, "SEA-Eval: 一个超越情景评估的自进化代理基准," 2026.[17] 张行, 王光辉, 崔延伟, 李子源, 邱伟, 朱冰, 贺沛阳, "谁来评判评判者？为自改进的大型语言模型代理共同进化的评估指标和技能," 2026.[18] 阿莫迪, 奥拉, 斯坦哈特, 克里斯蒂安诺, 舒尔曼, 和 丹·曼˙恩, "人工智能安全中的具体问题," 2016.[19] 奥伦, 梅斯特, 查特吉, 拉德哈克, 和 濑户部哲人, "证明黑盒语言模型的测试集污染," 2023.